

오늘의 작업기록 — 2026년 7월 4일

mini-ledger Layer 1 — 정산(이체 수수료) 구현

(솔로 런 복귀 — 야매는 7/2 하루로 끝. 그리고 돈 반올림이라는 핀테크 크럭스를 처음 마주한 날)

한눈에

항목	내용
목표	Layer 1 정산 — 이체에 수수료를 붙여 플랫폼 계좌로
결과	로직 완성 ✓ (Main 테스트만 WIP) — 수수료 계산·이동·기록 다 짤
커밋	add fee settlement logic (WIP, Main test pending) (2de4b3a)
저장소	github.com/Taewoo-Won/mini-ledger
오늘의 핵심	★ 돈 반올림 크럭스 (정수 나눗셈 합정) + 솔로 런 복귀
새 개념	정수 나눗셈이 소수점 버림 · 곱셈 먼저 나눗셈 나중 · 매직 넘버 → 필드 · 필드=동기화
방식	솔로 런 — 수수료 로직 전부 직접 작성, 문법 실수 단계별 교정
미완	Main 테스트(platform 계좌 생성 + 합 검증) 시간상 다음으로. 컴파일은 통과

환불 야매(7/2)를 7/3 복기로 갖고, 오늘 정산은 다시 솔로 런으로 돌아왔다. 설계 5개를 스스로 정하고, 수수료 로직을 직접 썼다. 문법 실수(변수 만들기 vs 메서드 호출 섞기, from/to 자리)를 여러 번 했지만 하나씩 잡으며 완성했다. 오늘의 진짜 수확은 기능보다 돈 반올림 — 정수 나눗셈이 소수점을 통째로 버린다는 걸 처음 마주한 것이다.

0. 출발점

- 이전까지: Layer 1 — 감사로그·명등성·조회·환불 완료. mini-ledger 4개 .java + 예외.
- 오늘 목표: 정산(이체 수수료) — 돈 보존(보낸 쪽 = 받은 쪽 + 플랫폼) 원칙 구현. 복식부기의 온램프.
- 다음: 복식부기 (Layer 1 마지막, 좀 무거움).
- 도구: IntelliJ(본진, 솔로 런 코딩) → 폰/VPS EOF 잔디.

PART 1 — 도메인: 돈은 사라지지 않는다

결제엔 수수료가 붙는다 — 1000 이체 시 플랫폼이 일부를 떼다(카드·송금 다 그렇다). 핵심 원칙:

돈은 사라지지 않는다. 수수료도 여딘가로 간다. 보낸 쪽에서 나간 돈 = 받은 쪽 + 플랫폼. 합이 보존돼야 한다.

→ 이게 복식부기("모든 돈 이동은 양쪽이 맞아떨어진다")의 온램프다. 완료 기준의 "세 잔액 합이 이체 전과 같음"이 곧 돈 보존 검증.

PART 2 — 설계 결정 5개 (솔로 런으로 스스로)

결정	내 답	근거
1. 누가 내나	보내는 쪽이 위에 없어서 추가 (1000 이체+수수료 별도)	받는 쪽은 보낸 만큼 온전히 = 송금 모델
2. 계산 방식	% (고정 아님)	실무 표준. 금액 클수록 수수료도 커야 공평
3. 수수료 목적지	platform 계좌 (createAccount 재사용)	없으면 돈 증발(6/23). 계좌 만드는 도구는 이미 있음
4. 기록	FEE enum 신설	TRANSFER로 뭉치면 일반 이체와 구분 불가(6/27 재구성 원칙)
5. 로직 위치	transfer 안	모든 이체에 수수료 자동. (환불은 transfer 안 쓰니 수수료 안 붙음 — 맞음)

→ 방식 1은 송금 모델(은행 이체 수수료 별도), 떼는 방식은 PG 정산 모델(결제액에서 땜). 내 프로젝트는 이체 중심이라 송금 모델(없기)이 적합.

PART 3 — ★ 오늘의 크럭스: 돈 반올림 (정수 나눗셈 합정)

1일차부터 돈은 long (정수 원), double 금지였다. 근데 %는 소수를 낳는다. 여기서 자바의 합정을 처음 마주했다.

3-1. 충격 — 2 / 100 = 0

요율 2%를 2 / 100 으로 쓰면? - 수학: 2 ÷ 100 = 0.02 - 자바: 2 / 100 = 0!

왜: 자바에서 정수 ÷ 정수 = 정수(뿔만, 소수점 버림). 2를 100으로 나누면 뿔 0, 나머지 2 → 0만 남는다.

```
2 / 100 → 0      (0.02 아님!)
99 / 100 → 0      (0.99 아님!)
7 / 2   → 3      (3.5 아님!)
```

→ 그래서 `amount * (2/100) = amount * 0 = 0`. 수수료가 항상 0. 회사 망하는 버그.

3-2. 해결 — 곱셈 먼저, 나눗셈 나중

```
long fee = amount * feeRate / 100; // 원→오 순서
= 1000 * 2 / 100
= 2000 / 100      ← 곱셈 먼저! 2000(정수)
= 20              ← 이제 나누면 20 ✓
```

괄호를 잘못 치면(`amount * (feeRate/100)`) 0, 순서대로(곱셈 먼저) 하면 정답. → 철칙: 돈 계산은 나눗셈을 마지막에. 큰 수 만들어놓고 마지막에 나눠야 소수점 버림 손실이 최소.

3-3. 남는 문제 — 반올림 (0.02원은 누가?)

1001의 2% = 20.02인데, 정수 나눗셈이라 `2002/100 = 20` (소수점 버림). **0.02원 사라짐**. - 수수료가 20.02 → 20으로 줄었으니 → **보내는 사람이 이득**(플랫폼 0.02 손해). - v1은 **버림(정수 나눗셈)**으로 채택 — 단순하고, 사용자 유리. (실무는 정책으로 올림/버림/반올림 정함. 면접 단골.)

오늘 제일 값진 건 이거다. "정수 나눗셈은 소수점을 통째로 버린다"는 걸 뼈에 새긴 것. 이자·수수료·할인 계산의 기본 함정이다.

PART 4 — 코드

4-1. transfer에 추가한 수수료 로직

```
public void transfer(String fromId, String toId, long amount, String idempotencyKey) {
    ... 기존 이체(a→b) + TRANSFER 기록 ...

    long fee = amount * feeRate / 100;           // 계산 (곱셈 먼저)
    fromAccount.withdraw(fee);                   // 보낸 쪽에서 수수료 더 빼기
    Account platformAccount = getAccount("platform"); // 플랫폼 계좌 꺼내기
    platformAccount.deposit(fee);                // 플랫폼에 수수료 넣기
    history.add(new Transaction(nextTxId++, TransactionType.FEE, fromId, "platform", fee)); // FEE 기록
    usedKeys.add(idempotencyKey);
}
```

- 필드: `private long feeRate = 2;` (final 안 붙임 — 요율은 바뀔 수 있어야)
- FEE 기록 from = `fromId` (누가 보냈든 그 사람), to = `"platform"` (항상 플랫폼)

4-2. 돈 흐름 (요율 2%, 1000 이체) — 예상

```
시작:  a=2100, b=3500, platform=0      (합 5600)
이체:   a -1000, b +1000
수수료: a -20, platform +20
끝:     a=1080, b=4500, platform=20     (합 5600) ← 보존 ✓
```

a가 총 1020 냄(이체+수수료), b가 1000 온전히 받음, platform 20. **합 5600 불변 = 돈 보존**. (※ Main 테스트로 눈 검증은 다음 세션 — 지금은 컴파일까지만.)

PART 5 — 오늘 잡은 개념들

5-1. 매직 넘버 → 필드로

코드에 그냥 박힌 숫자(`2`, `100`)를 **매직 넘버**라 함. 나쁜 이유: 뜻 불명확 + 여러 곳 흩어지면 바꾸기 지옥. → 의미 있는 값은 **이름 붙여 한 곳에**: `private long feeRate = 2;`. 요율 변경 = 이 한 줄만.

5-2. 필드 = 동기화 (원본 하나)

필드에 요율을 두면 = "**원본 한 곳**"이 생김. transfer 안 `feeRate` 는 값을 복사한 게 아니라 **필드를 참조**. → 필드를 3으로 바꾸면 그걸 쓰는 모든 곳이 자동으로 3을 씀(동기화). "박으면 못 바꾼다"가 아니라 "**필드로 빼야 한 곳에서 쉽게 바꾼다**."

5-3. final 판단

`feeRate` 엔 final 안 붙임 — 나중에 요율 바뀌어야 하니까. (usedKeys/refundedIds는 그릇 자체가 안 바뀌어 final 붙였지만, feeRate는 값이 바뀔 수 있어 안 붙임. **바뀔 값이면 final 빼기**.)

5-4. 문법 실수에서 배운 것 (솔로 런의 값)

직접 치며 낸 실수들 — 이게 학습이다: - 변수 만들기 ≠ 메서드 호출: `Account x = getAccount(...)` (만들기)와 `x.withdraw(fee)` (호출)는 다른 줄. 이미 있는 변수 (fromAccount)는 또 안 만들고 그냥 씬. - new Transaction엔 값만: `from "a"` 가 아니라 그냥 값(`fromId`). 자리(위치)가 역할을 정함. - 박힌 "a" vs fromId: 수수료 낸 사람은 "a" 고정이지 아니라 `fromId` (누가 보냈든). 매개변수 개념. - from/to 목적지 구분: 이체는 a→b(told), 수수료는 a→platform. 같은 transfer 안이어도 두 거래의 to가 다름.

6. 산출물

- 파일 2개 변경: `TransactionType.java` (FEE 추가), `Bank.java` (feeRate 필드 + transfer 수수료 로직).
- 커밋: `2de4b3a` "add fee settlement logic (WIP, Main test pending)". `85b3af6..2de4b3a` .
- 상태: 컴파일 통과. Main 실행 검증은 다음(platform 계좌 생성 + 합 보존 확인).
- 커밋 계보: `ee9f03b` (초회) → `44f09cf` (금액초회) → `85b3af6` (환불) → `2de4b3a` (정산 WIP)

7. 다음 작업

- Main 테스트 완성 — platform 계좌 생성 + 이체 + 세 잔액 합 보존 눈 검증. (오늘 못 한 것.)
- 복식부기 — 모든 거래가 양쪽 계정을 건드리게 (Layer 1 마지막).
- (선택) 반올림 정책 고도화(나머지 주처 % 연산).

8. 개념 사전

- 정수 나눗셈: 정수 ÷ 정수 = 몫(소수점 버림). `2/100=0` , `7/2=3` . 돈 계산의 핵심 함정.
- 곱셈 먼저, 나눗셈 나중: `amount * rate / 100` . 큰 수 만든 뒤 나눠야 소수점 손실 최소. 괄호로 나눗셈 먼저 하면(`amount*(rate/100)`) 0 됨.
- 매직 넘버: 코드에 박힌 의미 불명 숫자. 이름 붙여 필드/상수로 빼는 게 정석.
- 필드 참조 = 동기화: 필드 값 하나를 여러 곳이 참조. 필드만 바꾸면 다 반영.
- final: "한 번 정하면 안 바뀜". 바뀔 값(feeRate)엔 안 붙임.
- 송금 모델 vs PG 정산 모델: 수수료를 얻느냐(송금) 떼느냐(결제 정산). 이 프로젝트는 송금(얻기).
- FEE (enum 값): 수수료 거래임을 기록에 명시. TransactionType 6번째 값.

9. 비유 모음

- 정수 나눗셈 = 사탕 나누기: 사탕 2개를 100명이 나누면 1인당 0개(남는 2개는 못 나눔). 0.02개가 아니라 0개. 자바도 몫만.
- 곱셈 먼저 = 사탕 먼저 불리기: 2개를 1000배(2000개) 만든 뒤 100명이 나누면 1인당 20개. 순서를 바꾸면 나뉜다.
- 매직 넘버 vs 필드 = 벽에 낙서 vs 온도조절기: 온도를 벽 여기저기 써놓으면 바꿀 때 다 지워야. 조절기(필드) 하나면 돌리기만.
- platform 계좌 = 수수료 금고: 뎌 수수료가 같 곳. 없으면 돈이 허공으로(증발).

10. 검증 질문 (다음 복기용 — 답 적지 말 것)

- 자바에서 `2 / 100` 은 얼마인가? 왜? `99 / 100` 은?
- `amount * feeRate / 100` 을 `amount * (feeRate / 100)` 으로 바꾸면 결과가 어떻게 되나? 왜?
- `feeRate` 를 transfer 안에 숫자 `2` 로 박지 않고 필드로 뺀 이유는? (매직 넘버 / 동기화)
- `feeRate` 에 final을 안 붙인 이유는? usedKeys엔 붙였는데 왜 다른가?
- FEE 기록에서 from을 `"a"` 가 아니라 `fromId` 로 쓴 이유는?
- 수수료 기록의 to는 왜 `toId` 가 아니라 `"platform"` 인가? (이체와 수수료의 목적지 차이)
- 세 잔액 합이 이체 전과 같아야 하는 이유는? (돈 보존)

11. 회고

환불 야매(7/2)를 복기(7/3)로 갖고, 오늘 정산은 다시 솔로 런으로 돌아왔다. 설계 5개를 스스로 정하고 근거를 댔다 — 특히 "송금 모델(수수료 얻기)"을 고르고 이유를 설명한 게 도메인 감각이 붙은 증거다. 수수료 로직은 처음부터 끝까지 직접 쳤고, 문법 실수(변수 만들기과 메서드 호출을 한 줄에 섞기, from/to 자리 혼동)를 여러 번 냈지만 하나씩 잡으며 완성했다. 실수를 내고 고치는 이 과정이 솔로 런의 값이다 — 받아 적었으면 안 배웠을 것들이다.

오늘의 진짜 수확은 기능이 아니라 돈 반올림이다. `2 / 100 = 0` 이라는 걸 처음 보고 놀랐는데, 이게 핀테크에서 사람 잡는 함정이다. "곱셈 먼저, 나눗셈 나중"과 "정수 나눗셈은 소수점을 버린다"를 몸으로 새겼다. 이자·수수료·할인 어디서나 나오는 기본이고, 면접 단골이기도 하다.

아쉬운 건 시간이 부족해 Main 테스트를 못 한 것. 그래서 커밋도 WIP로 남겼다 — 컴파일은 되지만 실행으로 "합 보존"을 눈으로 확인하진 못했다. 이걸 다음 세션 첫 순서다. 미완을 미완이라고 정직하게 커밋 메시지에 남긴 건 잘한 거다. 로직 자체는 손에 있으니, platform 계좌 만들고 돌려보면 5600이 보존되는 걸 확인하게 될 것이다.

한 줄: 솔로 런으로 정산 수수료를 직접 짜며 `2/100=0` 이라는 핀테크 함정을 처음 마주하고 "곱셈 먼저"로 넘긴 날 — 기능보다 정수 나눗셈을 빼에 새긴 게 더 값지다.